Demonstration of General Accessibility Models: Learning Accessibility Needs from Everyday Computer Use

Yuxuan Liu
University of Michigan
Ann Arbor, USA
liurick@umich.edu

Ruijie Zheng
University of Michigan
Ann Arbor, USA
ruijiezh@umich.edu

Brandon Kim
University of Michigan
Ann Arbor, USA
kimdb@umich.edu

Jason Wu
Purdue University
West Lafayette, USA
jasonwu@purdue.edu

Anhong Guo
University of Michigan
Ann Arbor, USA
anhong@umich.edu

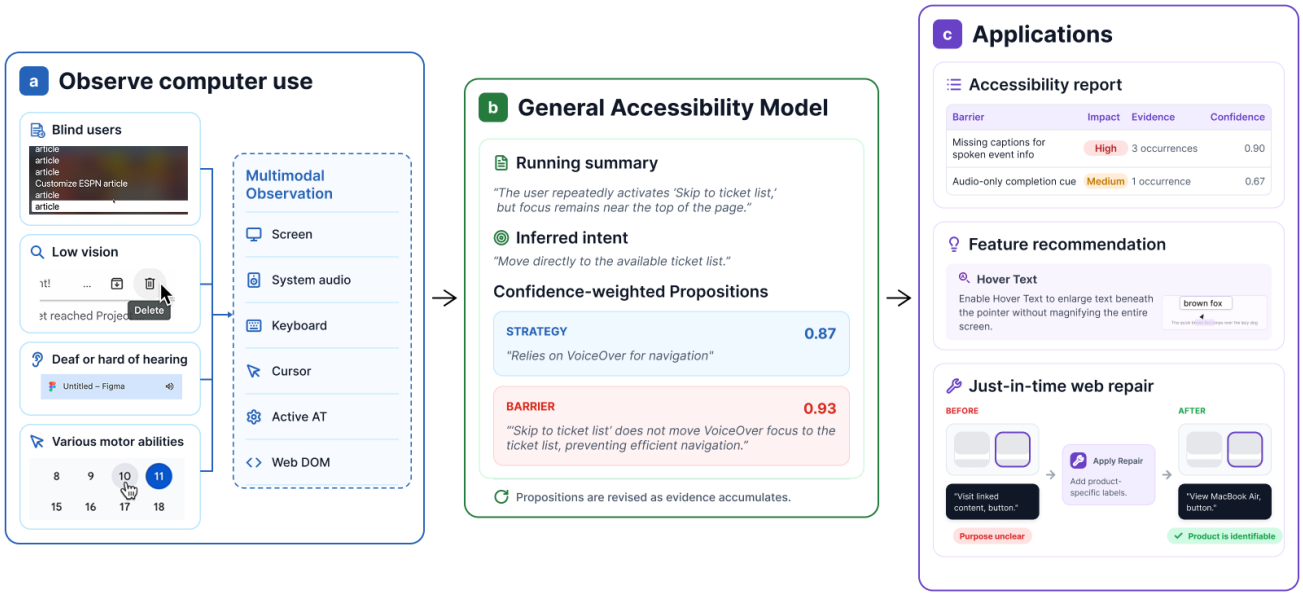

Figure 1: Overview of General Accessibility Model (GAM). (a) GAM observes everyday computer use by users with various abilities through synchronized multimodal observation. (b) GAM maintains a running summary, infers user intent, and updates confidence-weighted strategy and barrier propositions. (c) These propositions support accessibility reports, feature recommendations, and just-in-time accessibility repairs on Web.

Abstract

Accessibility research has long envisioned systems that adapt to users' abilities and contexts. Yet current ability-based systems lack reliable ways to learn users' accessibility needs in situ, leaving many individualized and context-specific barriers unaddressed. We present **General Accessibility Model (GAM)**, an architecture that learns user accessibility needs by observing everyday computer use. From screen recordings and input streams, GAM generates confidence-weighted propositions describing a user's interaction strategies and in-situ accessibility barriers. Because prolonged interactions are necessary to understand user needs, GAM employs efficient segmentation and context compression, periodically reflecting on observations to revise propositions. We demonstrate how GAM can enhance accessibility applications, including accessibility report generation, feature recommendation, and just-in-time accessibility repairs. GAM illustrates how everyday interaction can become a source of continuously updated knowledge for personalized accessibility support.

CCS Concepts

• **Human-centered computing** → **Accessibility systems and tools**.

Keywords

accessibility; assistive technologies; user modeling; UI generation; web accessibility; large multimodal models

1 Introduction

Computers are accessible when interfaces meet users' accessibility needs, yet this remains untrue for millions of people with disabilities. Automated audits found WCAG conformance failures [14] on 95.9% of the Web's one million most popular home pages [12], while many barriers fall beyond what guidelines encode [6, 7].

Existing approaches remain limited in detecting barriers as users encounter them. Automated checkers such as axe [2] and WAVE [11]

UIST Adjunct '26, Detroit, MI, USA
2026. ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

(a) Before repair (b) After repair

Figure 2: GAM's just-in-time Web repair workflow. (a) Before repair, VoiceOver announces each carousel item generically as "Visit linked content, button, group," leaving the item's destination unclear. (b) After GAM detects the repeated barrier and applies a repair to the carousel, VoiceOver announces a product-specific label, "Little Leaders Characters, button, group." The GAM extension panel shows the detected barrier and the enabled repair.

capture only a fraction of accessibility problems [10] and cannot determine whether a detected issue was actually experienced by a particular user. Many barriers are also dynamic, emerging only through the interaction among the user, the interface, and their assistive technology (AT).

Even when a barrier is known, existing approaches offer limited ways to mitigate it. Accessibility overlays apply generic transformations without knowing whether the transformation fits their needs [4]. Ability-based systems can personalize interfaces from measured abilities [13], but often rely on explicit elicitation or calibration tasks [3] that do not scale across everyday software or capture needs that shift with task and context.

Recent advances in multimodal LLMs create new opportunities for a situated and personalized approach. Computer-use agents demonstrate that these models can interpret screenshots and interact with graphical interfaces [1, 5]. General User Models (GUMs) further demonstrated that unstructured observations of computer use can be transformed into propositions about users' knowledge and preferences [8]. Together, these advances make it possible to model not only *what* users do, but also *how* they interact: whether an interface supports their goals, at what cost, and how they respond through particular strategies or ATs.

We present the **General Accessibility Model (GAM)**, an architecture that learns users' accessibility needs by observing everyday computer use. GAM segments and compresses continuous observations of the user's screen and input channels, then periodically reflects on accumulated evidence to create and revise confidence-weighted propositions describing interaction strategies and in-situ accessibility barriers, inspired by GUM's proposition-based representation. We illustrate how this representation supports accessibility report generation, accessibility feature recommendation, and Web interface repair, showing how everyday interaction can become continuously updated knowledge for personalized accessibility support. We plan to show all three applications at the demo booth.

2 General Accessibility Model (GAM)

GAM compares what the computer presents with what the user can access and do, then infers propositions about access strategies and barriers.

Observing computer use. In our current implementation, GAM runs on macOS and collects synchronized user interaction data that reveal what the user perceived, what they did, and what the interface exposed. It captures the screen recording and system audio, including AT output such as the VoiceOver cursor and speech, Zoom's magnified viewport, and Live Captions. It also records keyboard and cursor input to ground the model's interpretation in the user's actions, including screen-reader commands that never reach an application. GAM identifies active ATs through NSWorkspace accessibility queries and heuristic methods. On the Web, it additionally captures the page DOM to connect an experienced barrier to the elements that caused it.

From observations to propositions. Because evidence of accessibility needs may emerge across prolonged interaction, GAM processes observations incrementally. It divides the continuous stream at natural boundaries, such as pauses in input or navigation to a new page, while enforcing minimum and maximum segment durations. For each segment, Gemini 3.1 Pro receives the synchronized observations together with GAM's current context: a running session summary, the latest inferred user intent, and the existing propositions.

At the end of each segment, GAM creates or updates two types of confidence-weighted propositions. *Strategy propositions* describe recurring interaction patterns, such as "the user routinely relies on macOS Zoom at high magnification and pans across the viewport to inspect interface content." *Barrier propositions* describe accessibility problems encountered in a specific context, such as "the user's focus became trapped inside a modal dialog, preventing them from dismissing it or returning to the page content." or "during a file upload, the user missed an audio-only completion cue." As evidence accumulates, GAM can add, revise, reinforce, or remove propositions and update their confidence scores.

Each model call also updates the running summary and inferred intent. These outputs, together with the revised propositions, form the compressed context for processing the next segment.

3 Applications

GAM's propositions provide a shared representation of users' interaction strategies and encountered accessibility barriers that can support multiple downstream applications. Our demo will show

three applications that use this representation in different ways: accessibility report generation, accessibility feature recommendation, and just-in-time accessibility repair.

Accessibility report generation. Prior work such as AXNav has shown that automated accessibility testing can flag potential issues and produce artifacts for developer review [9]. Building on this direction, GAM's barrier propositions can be compiled into personalized accessibility reports for a specific website or application, describing the problems a user encountered and how they affected the user's task. Because these reports are grounded in observed interaction, they can capture dynamic and context-specific barriers that static audits may miss, providing developers with concrete evidence for reviewing and addressing problems difficult to detect. For example, a report generated from the interactions of a deaf or hard-of-hearing user might include missing captions for spoken information and alternative forms for audio-only completion cue.

Accessibility feature recommendation. Prior work has explored recommending accessibility features from observed device-use patterns [15]. GAM builds on this direction by grounding recommendations in patterns across strategy and barrier propositions. For example, if a low-vision user repeatedly magnifies small portions of a page and pans the viewport to inspect text, GAM could recommend macOS Hover Text, which enlarges the text beneath the pointer without magnifying the entire screen. GAM could also recommend more specific uses of accessibility features. For example, if a blind user is unfamiliar with a website and repeatedly navigates sequentially and backtracks, GAM could recommend relevant VoiceOver commands, such as moving by headings or jumping to the next link, to help them reach their goal more efficiently.

Just-in-time accessibility repair. For accessibility barriers associated with website, GAM supports just-in-time interface repair. A Chrome extension presents each detected barrier retrieved from GAM and lets the user request a repair. The repair module inputs the barrier description with technical details and relevant DOM context to a LLM to generate a JavaScript that patches the affected elements. Figure 2 shows an example in which VoiceOver announces every carousel item only as "Visit linked content, button, group," making it unclear which item each control opens. GAM detects this repeated barrier and generates a repair that assigns each carousel item a distinct accessible name based on its image content. After the repair is applied, VoiceOver announces the focused item as "Little Leaders Characters, button, group," allowing the user to identify it. Users can also enable repairs to be applied automatically on future visits to the same URL.

References

[1] Anthropic. 2024. Introducing Computer Use, a New Claude 3.5 Sonnet, and Claude 3.5 Haiku. https://www.anthropic.com/news/3-5-models-and-computer-use. Accessed July 2026.

[2] Deque Systems. 2026. axe-core: Accessibility engine for automated Web UI testing. https://github.com/dequelabs/axe-core. Accessed July 2026.

[3] Krzysztof Z. Gajos, Jacob O. Wobbrock, and Daniel S. Weld. 2008. Improving the Performance of Motor-Impaired Users with Automatically-Generated, Ability-Based Interfaces. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '08)*. ACM, 1257–1266.

[4] Tlamelo Makati, Garreth W. Tigwell, and Kristen Shinohara. 2024. The Promise and Pitfalls of Web Accessibility Overlays for Blind and Low Vision Users. In *Proceedings of the 26th International ACM SIGACCESS Conference on Computers and Accessibility (ASSETS '24)*. ACM. doi:10.1145/3663548.3675650

[5] OpenAI. 2025. Introducing Operator. https://openai.com/index/introducing-operator/. Accessed July 2026.

[6] Helen Petrie and Omar Kheir. 2007. The Relationship between Accessibility and Usability of Websites. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '07)*. ACM, 397–406.

[7] Christopher Power, André Freire, Helen Petrie, and David Swallow. 2012. Guidelines Are Only Half of the Story: Accessibility Problems Encountered by Blind Users on the Web. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '12)*. ACM, 433–442.

[8] Omar Shaikh, Shardul Sapkota, Shan Rizvi, Eric Horvitz, Joon Sung Park, Diyi Yang, and Michael S. Bernstein. 2025. Creating General User Models from Computer Use. In *Proceedings of the 38th Annual ACM Symposium on User Interface Software and Technology (UIST '25)*. ACM, 1–23. doi:10.1145/3746059.3747722

[9] Maryam Taeb, Amanda Swearngin, Eldon Schoop, Ruijia Cheng, Yue Jiang, and Jeffrey Nichols. 2024. AXNav: Replaying Accessibility Tests from Natural Language. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems (CHI '24)*. Association for Computing Machinery, New York, NY, USA. doi:10.1145/3613904.3642777

[10] Markel Vigo, Justin Brown, and Vivienne Conway. 2013. Benchmarking Web Accessibility Evaluation Tools: Measuring the Harm of Sole Reliance on Automated Tests. In *Proceedings of the 10th International Cross-Disciplinary Conference on Web Accessibility (W4A '13)*. ACM.

[11] WebAIM. 2026. WAVE Web Accessibility Evaluation Tools. https://wave.webaim.org/. Accessed July 2026.

[12] WebAIM. 2026. The WebAIM Million: The 2026 report on the accessibility of the top 1,000,000 home pages. https://webaim.org/projects/million/. Accessed July 2026.

[13] Jacob O. Wobbrock, Shaun K. Kane, Krzysztof Z. Gajos, Susumu Harada, and Jon Froehlich. 2011. Ability-Based Design: Concept, Principles and Examples. *ACM Transactions on Accessible Computing* 3, 3 (2011). doi:10.1145/1952383.1952384

[14] World Wide Web Consortium. 2023. Web Content Accessibility Guidelines (WCAG) 2.2. https://www.w3.org/TR/WCAG22/. W3C Recommendation.

[15] Jason Wu, Gabriel Reyes, Sam C. White, Xiaoyi Zhang, and Jeffrey P. Bigham. 2021. When Can Accessibility Help?: An Exploration of Accessibility Feature Recommendation on Mobile Devices. In *Proceedings of the 18th International Web for All Conference (W4A '21)*. Association for Computing Machinery, New York, NY, USA, 1–12. doi:10.1145/3430263.3452434